Kiểm chứng cuối cho sàng lọc tính khả thi robust SINR
Đánh giá runtime, kiểm thử biên và dữ liệu kênh độc lập

Tài liệu kỹ thuật và hướng dẫn tái lập – 10/11/2026

Abstract

Trong đường xuống đa người dùng nhiều anten (MU-MIMO), trạm gốc lựa chọn tập người dùng và hướng tạo búp sao cho các ràng buộc tỷ số tín hiệu trên giao thoa cộng nhiễu (SINR) được đáp ứng dưới sai số thông tin trạng thái kênh (CSI). Kết quả R2.V11 đã xác nhận bảo toàn nghiệm trong tập ứng viên hữu hạn và giảm số lời gọi bộ giải, nhưng screening làm tăng runtime trung bình. Gói này triển khai một kiểm chứng tiếp theo nhằm xác định liệu kiểm tra nhanh các điều kiện cần về tính khả thi có đem lại lợi ích runtime riêng hay không. Mọi kết quả thất bại hoặc chưa đủ bằng chứng đều phải được báo cáo; bản thảo chỉ được xét khóa khi có kết quả kiểm chứng độc lập đáp ứng điều kiện quy định trước.

I. Bài toán và phạm vi cố định

Trạm gốc có $M = 16$ anten, xét $K = 4$ người dùng và $L = 7$ bộ hướng tạo búp zero-forcing (ZF) hoặc regularized zero-forcing (RZF) tính lại cho mỗi tập con. Mỗi cấu hình xét $L(2^K - 1) = 105$ ứng viên. Giữ nguyên $q_c = 2.960414$, sai số CSI tương đối 0.04, SINR mục tiêu 6 dB, công suất nhiễu $2 \cdot 10^{-13}$ W và $P_{\max} = 1$ W. Các thông số phải được kiểm tra phù hợp với chuẩn hóa kênh thực tế trước khi mở dữ liệu kiểm thử. Không tối ưu CSI margin hoặc hệ số RZF bằng tập kiểm thử.

Với tập người dùng $\mathcal{S}$ và một hướng beam đã cố định, điều kiện robust SINR có dạng

$$\mathbf{p} \geq \mathbf{F}\mathbf{p} + \mathbf{u}, \qquad \mathbf{F} \geq 0, \quad \mathbf{u} > 0, \quad \mathbf{1}^T\mathbf{p} \leq P_{\max}. \tag{1}$$

Phương trình (1) xác định yêu cầu công suất khi giao thoa giữa những liên kết đã được chấp nhận phải được kiểm soát. Bài toán ưu tiên tối đa số người dùng phục vụ đồng thời, sau đó tối thiểu tổng công suất trong số phương án có cùng số người dùng. Exhausive search trên tập hữu hạn được dùng làm oracle; không suy diễn tối ưu toàn cục trên mọi beam phức liên tục.

II. Cơ chế screening và nguyên tắc đo công bằng

Cận do nhiễu $\mathbf{1}^T\mathbf{u} > P_{\max}$ và cận tổng riêng Neumann

$$L_r = \mathbf{1}^T \sum_{n=0}^{r} \mathbf{F}^n \mathbf{u} > P_{\max}, \quad r \leq 4, \tag{2}$$

cho phép loại ứng viên trước khi giải đầy đủ, vì L_r là cận dưới tổng công suất của mọi nghiệm đáp ứng (1). Với cặp (i, j), điều kiện $F_{ij}F_{ji} > 1$ cũng đủ loại ứng viên theo cấu trúc ma trận giao thoa không âm. Trường hợp độ lợi trực tiếp robust không dương được giữ lại để giải đầy đủ nếu gần ngưỡng số học.

Đặt $\tau = 10^{-5}$ là khoảng bảo vệ tương đối. Nếu khoảng cách của bất đẳng thức tới ngưỡng không vượt τ, ứng viên không bị loại (*fail-open*). Bước này không phải chứng nhận interval arithmetic cho toàn bộ phép tính kênh và beam.

Đối chứng `CARDINALITY_ONLY` gọi trực tiếp bộ giải nguồn, không chịu chi phí dựng ma trận cho screening. `FAST` sử dụng bốn kiểm tra trên, còn `LITE` bỏ cận Neumann để đánh

giá overhead. Cả ba cùng sử dụng hàm giải công suất và cùng tập ứng viên. Runtime đo từ đầu đến cuối mỗi lần lựa chọn, bao gồm sinh beam, screening và bộ giải, không đưa runtime oracle vào runtime thuật toán. Đảo thứ tự các phương pháp giữa các lượt lặp để giảm sai lệch do thứ tự chạy.

III. Thiết kế dữ liệu kiểm chứng

DeepMIMO: Nhập $\widehat{\mathbf{H}}$ có kích thước $(N, M, 4)$ dưới dạng số phức, trường `groups` và metadata tọa độ/scenario. Dữ liệu kiểm thử phải thuộc nhóm không gian chưa xuất hiện trong phát triển v9–v11, được ghi trong `provenance.json`. Đối chiếu tập nhãn nhóm phát triển bằng tùy chọn `--development-groups`. Việc không trùng mã nhóm chỉ là phép kiểm tra tối thiểu; không chứng minh độc lập thống kê.

3GPP TR 38.901: Xuất kênh từ mô phỏng Sionna UMi hoặc kịch bản 3GPP đã cấu hình thực sự, lưu thông tin scenario, tần số, vận tốc, seed, độ trễ, chuẩn hóa và noise model. Không sử dụng bộ tạo `umi-inspired` rút gọn để công bố là kết quả 3GPP. Bản mã nhận kênh qua NPZ, không tự tạo dữ liệu chuẩn. Các phép so sánh phải dùng cùng quy trình cho cả hai nguồn và giữ nguyên tham số đã khóa.

IV. Điều kiện dừng và kiểm định

Tại mỗi geometry, nghiệm FAST, LITE và CARDINALITY_ONLY phải trùng oracle về số người dùng, tập người dùng, lựa chọn beam và công suất trong dung sai số học đã định sẵn. Mỗi ứng viên bị loại bằng FAST được so sánh với exhaustive; một trường hợp loại nhầm là FAIL. Chênh lệch runtime chính là $\Delta t = t_{\mathrm{CARD}} - t_{\mathrm{FAST}}$, tính theo trung vị những lần chạy lặp của cùng geometry. Bootstrap ghép cặp theo nhóm không gian cho khoảng tin cậy 95% mô tả. Chỉ xét đủ điều kiện GO riêng cho FAST khi cận dưới khoảng này lớn hơn 0; kết luận chính thức đòi hỏi cả hai nguồn độc lập đều đạt. Quyết định trên tập test tuyệt đối không được dùng để lựa chọn lại quy tắc hoặc tham số rồi tuyên bố cùng tập đó vẫn là holdout.

V. Checkpoint và danh sách tệp

Mốc	Tệp kết quả	Kiểm tra bắt buộc
CP00	`CP00_frozen_protocol.json`	Hash mã, tập beam, ngưỡng, quy tắc GO
CP01	`CP01_dataset_audit.json`	SHA-256 dữ liệu, metadata, tách nhóm
CP02	`CP02_repeated_runtime.csv`	Từng geometry, lần lặp, runtime, lời gọi giải
CP03	`CP03_all_candidate_audit.csv`	Từng ứng viên, certificate và oracle
CP04	`CP04_paired_geometry.csv`	Thời gian tiết kiệm và lời gọi giải ghép cặp
CP05	`CP05_comparison_summary.csv`	Runtime, khoảng tin cậy, FAST/LITE
CP06	`CP06_decision.json`	PASS/STOP và giới hạn kết luận
CP07	`CP07_SHA256.csv`	Chữ ký của các checkpoint
B-CP00–03	`boundary_stress.py`	Các thử nghiệm sát ngưỡng và fail-open
Final	`manuscript_gate.py`	Tổng hợp quyết định ba nhánh độc lập

Mỗi geometry có tệp JSON trong `outputs/RUN_ID/tasks/`. Tệp được ghi bằng tệp tạm, `fsync` và thay thế nguyên tử. Chạy lại cùng lệnh `scan` sẽ bỏ qua geometry hoàn thành;

chữ ký mismatch làm dừng, yêu cầu run-id mới. Chạy `report` chỉ khi toàn bộ geometry đã hoàn thành. Không chép các kết quả smoke test vào các bảng chính của manuscript.

VI. Lệnh chạy Windows PowerShell

Mở PowerShell trong thư mục gốc của gói. Trước khi dùng dữ liệu thực, cài đặt thư viện và chạy unit test:

```
python -m pip install -r code/requirements.txt
python -m unittest discover -s tests -v
```

Kiểm tra sát ngưỡng. Không sử dụng tập test độc lập để chọn lại ngưỡng:

```
python code/boundary_stress.py --stage all \
  --run-id BORDER_V12 \
  --dataset data/development_stress.npz --limit 50
```

Chương trình `scripts/run_validation.ps1` thực hiện tuần tự `lock`, `preflight`, `scan`, `report`; nếu đã có checkpoint hợp lệ, chạy lại cùng lệnh sẽ tiếp tục phần chưa hoàn thành.

```
.\scripts\run_validation.ps1 `
  -Source DeepMIMO `
  -Dataset data\deepmimo_new.npz `
  -Provenance data\deepmimo_provenance.json `
  -DevelopmentGroups data\development_groups.json `
  -RunId DM_V12
```

Đối với kênh 3GPP độc lập, đổi `-Source` thành `Sionna_3GPP`, đổi đường dẫn dataset và provenance, đồng thời sử dụng run-id `GPP_V12`. Giữ nguyên thuật toán và tham số vật lý. Nếu chính sách thực thi PowerShell không cho phép chạy script, dùng `powershell -ExecutionPolicy Bypass -File scripts/run_validation.ps1` cùng các tham số trên.

Ghi lại phần cứng, số luồng, phiên bản Python/NumPy/SciPy và tải hệ thống. Script đặt số luồng BLAS và OpenMP bằng 1 cho từng lần chạy. Không thay đổi mã hoặc dữ liệu trong một run-id đã khóa.

Chỉ thực hiện cổng quyết định khóa bản thảo sau khi cả ba nhánh đã chạy xong:

```
python code/manuscript_gate.py `
  --deepmimo-dir outputs/DM_V12 `
  --threegpp-dir outputs/GPP_V12 `
  --boundary-dir outputs/BORDER_V12 `
  --out outputs/manuscript_gate.json
```

VII. Kết quả kiểm tra chương trình và kết luận

Bộ mã đã vượt 12 bài unit test và một smoke test 4 geometry/420 ứng viên với nghiệm khớp exhaustive; đây là kiểm tra tính đúng chương trình, không phải kết quả khoa học. Chưa có kết quả independent DeepMIMO hoặc 3GPP chính thức trong gói này. Không sửa Abstract/Conclusion để tuyên bố tăng tốc trước khi cổng khóa bài đáp ứng. Nếu FAST không giảm runtime so với cardinality-only trên dữ liệu độc lập, phải giữ kết luận về tính đúng của screening, nhưng dừng tuyên bố về lợi thế runtime.